CONTENT

	PAGE No.
Abstract	i
List of Figures	ii
List of Tables	iii
List of Acronyms	iv
CHAPTER I	
INTRODUCTION	01 – 05
1.1 Overview	01
1.2 Motivation	01
1.3 Problem Statement	02
1.4 Objective	02
1.5 Significance	03
1.6 Applications	03
1.7 Project Contribution	04
1.8 Project Layout	04
CHAPTER II	
LITERATURE SURVEY	06 – 08
CHAPTER III	
TRADITIONAL SYSTEM	09 – 10
3.1 Traditional System	09
3.2 Limitations	10
CHAPTER IV	
PROPOSED SYSTEM	– 24
4.1 Overview	11
4.2 Data Preprocessing	14
4.3 SMOTE	15
4.4 Train Test Splitting	16
4.5 ML Model Build and Train	17
4.5.1 K-Nearest Neighbors (KNN) Classifier	18
4.5.2 K-Nearest Neighbors Classifier Limitations	21
4.5.3 Extra Tree Classifier	21
4.5.4 Extra Trees Classifier Advantages	24

CONTENT

CHAPTER V

SOFTWARE REQUIREMENTS – 28

5.1 Software Requirements 25

5.2 Hardware Requirements 27

CHAPTER VI

SOURCE CODE – 33

CHAPTER VII

RESULTS AND DESCRIPTION – 42

7.1 Implementation Description 34

7.2 Dataset Description 37

7.3 Result Analysis 38

7.4 Comparative Analysis 40

CHAPTER VIII

CONCLUSION AND FUTURE SCOPE 43

8.1 Conclusion 43

8.2 Future Scope 43

REFERENCE 44- 45

ABSTRACT

Maternal health is a critical component of public health, with early detection of risks playing a pivotal role in ensuring the safety and well-being of both the mother and the fetus. This study presents a data-driven approach to Maternal Health Risk Prediction, leveraging Internet of Things (IoT) sensor data and Machine Learning (ML) to facilitate timely intervention and improve prenatal care outcomes. The integration of IoT technology enables real-time monitoring of vital health parameters such as blood pressure, blood sugar, body temperature, and heart rate, thereby generating a continuous stream of rich physiological data. The study initially employed the K-Nearest Neighbors (KNN) Classifier as the baseline model due to its simplicity and effectiveness in handling small to moderate datasets. However, limitations such as sensitivity to feature scaling and computational inefficiency with large data volumes necessitated the development of a more robust alternative. As a result, the Extra Trees Classifier is proposed as the advanced model, offering improved accuracy and efficiency through ensemble learning, randomized feature splits, and enhanced handling of high-dimensional data. Comparative evaluation of both models was conducted using key performance metrics such as accuracy, precision, recall, and F1score. The Extra Trees Classifier demonstrated superior predictive performance and generalization ability, making it a more suitable choice for early risk classification in maternal healthcare applications.

LIST OF FIGURES

Name of the Figure **Page No.**

Fig 4.1: Proposed Block Diagram 12

Figure 4.2: Splitting the dataset. 17
Figure 4.3: K-Nearest Neighbors Classifier Block Diagram 19
Figure 4.4: Extra Trees Classifier Block Diagram 22
Fig 7.1 : Uploading Dataset 37
Fig 7.2 : Correlation heatmap 38
Fig 7.3 : Countplot of Target Column 39
Fig 7.4 : Confusion matrices of Existing KNN and proposed ETC 40
Fig 7.5 Prediction obtained on test data using proposed ETC 40

LIST OF TABLES

Name of the Table **Page No**.

Table.7.1 Performance Comparison of Various Algorithms Performance Comparison Table: Existing KNN vs. Proposed ETC 40

LIST OF ACRONYMS

Acronym		Full Form
AI	:	Artificial Intelligence (implied from context with ML/IoT)
API	:	Application Programming Interface (possible, not explicitly stated)
CSV	:	Comma-Separated Values
EDA	:	Exploratory Data Analysis
EHR	:	Electronic Health Records
ETC	:	Extra Trees Classifier
GRU	:	Gated Recurrent Unit
IoT	:	Internet of Things
KNN	:	K-Nearest Neighbors
LMT	:	Logistic Model Tree
LSTM	:	Long Short-Term Memory
ML	:	Machine Learning
RFE	:	Recursive Feature Elimination
RF	:	Random Forest
ROC-AUC	:	Receiver Operating Characteristic - Area Under Curve
SMOTE	:	Synthetic Minority Oversampling Technique
SVM	:	Support Vector Machine

CHAPTER 1 INTRODUCTION

1.1 Overview

Maternal Health Risk (MHR) refers to potential health problems arising during pregnancy, childbirth, and postpartum. According to WHO, there are around 280,000 fatalities of women due to pregnancy complications, which means a woman dies approximately every two minutes (WHO, 2023). The various factors increase the mortality rate of maternal women and childbirth, including the shortage of doctors and nurses and the localization, time, and distance According to WHO's report in 2020, around 800 women die daily due to poor resources and care .Despite recent technological advances, the rate of maternal death is decreasing, making it difficult to ensure both the mother's and child's safety during pregnancy. Pregnancy-related risks can be reduced in this scenario by anticipating complications and taking precautions. Some studies have been conducted in recent years to predict certain risks that can occur during pregnancy and to predict the birth method best suited to mothers' pregnancy characteristics.

1.2 Motivation

Predicting MHR aims to improve the overall health of pregnant women and their babies. MHR can occur during pregnancy, childbirth, and the postpartum period. However, it is most prevalent during pregnancy when women are at a higher risk of developing health issues, which can lead to miscarriage and death in certain circumstances (Hussain et al., 2014). By identifying and assessing the potential health risks early on, healthcare professionals can take measures to prevent, manage, or treat these conditions. Predicting health risks can also help healthcare systems to allocate resources more effectively. Healthcare providers can prioritize their care by identifying women at higher risk. It can also empower pregnant women with information about their risk factors and allow them to make informed decisions about their health. . The study aims to develop an Android application that integrates with IoT devices, such as wearable sensors and remote monitoring systems, to predict and mitigate maternal health risks that arise during pregnancy.

1.3 Problem Statement

Maternal health is a critical aspect of public healthcare, yet millions of women worldwide still face risks during pregnancy due to late diagnosis and inadequate monitoring. Many complications, such as preeclampsia, gestational diabetes, and fetal distress, can develop silently, making early detection essential. Traditional healthcare approaches rely on periodic check-ups, which may not always capture real-time health fluctuations. For expectant mothers in remote areas or those with limited access to healthcare, timely intervention becomes even more challenging. As a result, preventable complications often escalate, leading to increased maternal and infant mortality rates. With the rise of **IoT-enabled wearable devices** and **machine learning-driven health analysis**, there is an opportunity to bridge this gap. IoT devices can continuously track vital signs like blood pressure, heart rate, glucose levels, and fetal movements, providing a steady stream of real-time health data. However, without an intelligent system to process and interpret this data, the information remains underutilized. Machine learning can classify risks, predict complications early, and alert healthcare providers for timely intervention. The challenge lies in developing accurate, reliable, and interpretable ML models that can seamlessly integrate with IoT devices and provide actionable insights. Addressing these challenges will enable **proactive maternal care**, reducing risks and ensuring safer pregnancies.

1.4 Objective

The primary objective of this study is to develop an intelligent and data-driven system that can accurately predict maternal health risks using IoT-based sensor data and machine learning techniques. By leveraging real-time monitoring of vital parameters such as blood pressure, heart rate, blood sugar, and body temperature, the system aims to detect potential health risks at an early stage, enabling timely medical intervention. This research also seeks to design a reliable and scalable machine learning model that can be integrated with wearable devices and healthcare systems, providing continuous support throughout pregnancy. Furthermore, the study aims to compare traditional models like K-Nearest Neighbors (KNN) with more robust ensemble techniques such as the Extra Trees Classifier, identifying the most efficient approach for multiclass risk classification. In addition to predictive accuracy, the objective includes improving the interpretability and usability of the system, making it practical for both healthcare professionals and expectant mothers.

1.5 Significance

The significance of this study lies in its potential to revolutionize maternal healthcare through the integration of IoT and machine learning technologies. Traditional prenatal care often relies on periodic check-ups, which may not capture sudden or gradual health changes. This research introduces a continuous, intelligent monitoring system capable of identifying risks in real-time, significantly reducing the chances of undetected complications. For populations in remote or underserved regions, where access to healthcare is limited, the proposed system offers a practical solution by providing remote risk assessment and timely alerts. Economically, the solution contributes to reducing maternal care costs by minimizing emergency interventions and unnecessary hospital visits. Technologically, it pushes the boundary of smart healthcare systems by merging real-time IoT data with advanced machine learning models. This not only enhances decision-making for healthcare professionals but also empowers pregnant women with knowledge about their health, fostering preventive care. Overall, the study addresses critical challenges in maternal health, aiming to reduce maternal mortality and improve outcomes for mothers and their babies.

1.6 Applications

- **Continuous Maternal Health Monitoring** – IoT wearables track vital signs like blood pressure, heart rate, and glucose levels, providing real-time health insights.
- **Early Risk Detection and Alerts** – Machine learning models analyze data to predict complications such as preeclampsia or gestational diabetes, alerting doctors instantly.
- **Remote Healthcare Assistance** – Expectant mothers in remote areas receive AI-driven health assessments, reducing the need for frequent hospital visits.
- **Personalized Pregnancy Care Plans** – AI tailors recommendations based on individual health patterns, optimizing diet, medication, and activity levels.
- **Emergency Response Optimization** – Real-time alerts help hospitals and emergency teams prepare for high-risk deliveries, improving response time and care quality.

- **Integration with Telemedicine** – IoT and ML enhance virtual consultations by providing doctors with real-time patient data for accurate remote diagnosis and treatment.

1.7 Project Contribution

The project contributes in several technical and practical aspects:

1. **IoT-Based Real-Time Monitoring**
   Vital signs such as blood pressure, heart rate, and body temperature are collected using IoT wearables for real-time maternal health tracking.
2. **Machine Learning Model Comparison**
   Compares K-Nearest Neighbors (KNN) and Extra Trees Classifier for risk prediction, proving that Extra Trees provides superior accuracy and efficiency.
3. **Preprocessing and Class Balancing**
   Extensive data preprocessing using label encoding, SMOTE for class balancing, and exploratory data analysis to enhance model robustness.
4. **Android Integration Possibility**
   Indicates development towards an Android app for continuous monitoring and alerts, offering scalable use in remote and underserved areas.
5. **Automation of Maternal Risk Prediction**
   Enables proactive intervention through intelligent classification of Low, Mid, and High risk levels, moving away from reactive healthcare systems.

1.8 Project Layout

The **first chapter** of the project, titled Introduction, lays the foundation by outlining the overall objective and motivation behind the study. It highlights the importance of maternal health and the risks associated with pregnancy, childbirth, and the postpartum period. The chapter discusses how integrating IoT and machine learning technologies can revolutionize maternal healthcare by enabling early risk detection. It also includes a clear problem statement, the specific goals of the project, its significance in addressing healthcare gaps, and potential real-world applications of the proposed system.

The **second chapter**, Literature Survey, reviews related research in the field of maternal health monitoring using machine learning and IoT. It summarizes various approaches

and methodologies adopted by previous researchers, such as ensemble models, neural networks, and data balancing techniques like SMOTE. This chapter provides insight into how existing work has tackled similar problems and identifies the gaps that this project aims to fill.

The **third chapter**, Problem Definition, focuses on the shortcomings of traditional maternal health monitoring systems. It outlines the manual data collection process, subjective diagnosis by healthcare professionals, and the lack of real-time monitoring. The limitations such as delayed detection, static threshold-based analysis, and lack of personalization emphasize the need for a more intelligent and automated solution.

The **fourth chapter**, System Design, details the proposed solution. It starts with an overview of the IoT-based health data collection process and describes how the collected data is preprocessed and analyzed. The chapter explains the steps of applying feature engineering, handling class imbalance with SMOTE, and implementing two machine learning models—K-Nearest Neighbors (KNN) and Extra Trees Classifier (ETC). It concludes with a comparison of these models based on metrics like accuracy, precision, recall, and F1-score, showing the superior performance of the ETC model.

The **fifth chapter**, Implementation Issues, covers the technical setup required to build the system. It lists the software and hardware prerequisites, including specific Python versions and libraries like scikit-learn, pandas, and imbalanced-learn. It also describes the system configuration and how the models are trained, validated, and saved for reuse.

The **sixth chapter**, Conclusion and Future Scope, summarizes the project's achievements and highlights the successful development of a machine learning-based maternal health risk prediction system. The chapter outlines the advantages of the Extra Trees model and its integration with IoT for real-time monitoring. It also discusses potential improvements, such as incorporating real-time data streaming, using deep learning techniques like LSTM or GRU, and developing a mobile app for wider accessibility.

CHAPTER 2

LITERATURE SURVEY

This section demonstrates a few related kinds of literature conducted before using approaches like Neural Networks (NN), ML classifiers, and the ensemble technique to combine the different architectures for predicting maternal health risk factors. Some of the studies focus on monitoring systems during pregnancy time. Ali Raza et al. proposed an ensemble method, BiLTCN that combined the NN-based BiLSTM, Temporal Convolutional Network, and Decision Tree as a classifier using the clinical dataset of 1218 instances collected by the IoT-enabled system. The proposed system observed results after balancing using SMOTE with an average accuracy of 88%. Also, they applied feature selection techniques and used SVM along with BiLTCN, claiming 98% accuracy on the reduced feature model. Ahmed et al. executed research by using the ML models and concluded that the Logistic Model Tree (LMT) classifier performs better in analyzing the factors related to maternal health. The IoTenabled system data were collected and deployed on the LMT model, producing 90% accuracy.

The mortality prediction rate was developed using the ML models, and the twoclass SVM model produced a more accurate accuracy of 86.7% compared to other models Rani and Kumar. Also, Akbulut et al developed the fetal health monitoring system using the Decision Forest Model with an accuracy of 89.5% under test conditions compared to other ML models. Sarhaddi et al. proposed an IoT-based Maternal health monitoring system for long-term uses that monitor pregnant women the entire time. Assaduzzaman et al. focused on ML model to develop risk factors for maternal health using a dataset that preprocessed and applied feature engineering techniques to develop a prediction model using RF and other ML classifiers; among them, RF achieved an accuracy of 90% which was a most top model. Pereira et al. addressed the health monitoring system of maternal risk factors using six ML models and applied the feature elimination technique RFE to the feature set. The RF classifier with RFE achieved the highest mean accuracy of 93.24%. Pawar et al. deployed eight ML models using the k-fold cross-validation technique to classify maternal risk into three classes. Among the models, RF provided the best results, with a mean accuracy of 70.21%. Maternal health risk prediction aims to develop and

implement models and systems that can effectively predict the risk associated with maternal health outcomes during pregnancy.

It involves research, data collection, model development, result validation, and implementation to improve maternal health care and reduce mortality rates. The concepts used in this study are ML, IoT, and Software Development (Android application). ML techniques have an important role in maternal health risk prediction. It has been widely used in predicting the mode of childbirth and assessing the potential maternal risk during pregnancy. These techniques allow us to develop prediction models to analyze data and identify patterns, correlations and predictive factors that give rise to adverse maternal health outcomes. Machine Learning can be utilized through Data Analysis and Feature Selection, Model Development, Training and Validation, and Predictive analysis.

The classification task of predicting a specific disease, malware, or conditions using ML techniques enables one to reduce the dimension of the features using feature selection techniques or applying the data analysis approaches and combining the different model's predictions using ensemble techniques Islam et al. The upcoming challenges in the medical field are the development of modern IoT devices and the environment provided by the technology enhancement and the uses of IoT applications. With the recent development of the new Medical 4.0 in the healthcare sector, everything is now connected through IoT nodes, even hospital beds, to patients' physical and biological characteristics. The application of Medical 4.0 in healthcare sectors is discussed by Haleem et al. and provides the details to decrease the cost of healthcare expenses in underdeveloped or developed countries. Patient data is digitalized, and the transformation of doctor-centric treatment at a hospital or clinic is replaced by IoT technology to patient-centric approaches. Medical 4.0 is embedded with industry 4.0 at the manufacturing level with high safety, security and privacy and is more effective Oliveira et al. Al-Jaroodi et al., The IoT has a significant role in maternal health risk prediction. It can provide real-time monitoring, data collection, and connectivity between devices. In the research study, three types of IoT devices (Heart rate, blood pressure, and body temperature measuring) will be used; these devices will provide real-time data for risk assessment. Many IoT-based software applications are developed to increase the satisfaction level of patients through smooth communication among the

hospitals and are always connected through IoT-enabled applications regardless of the physical locations (Pang et al.; Gupta et al. Celdrán et al. Jaleel et al. From the above analysis, we note that there is a lack of work on automatic health risk prediction and monitoring of a woman during their maternal. Therefore, the proposed work is important because it integrates IoT and ML to automatically diagnose abnormalities of a woman during their maternal smart at early stage.

CHAPTER 3 TRADITIONAL SYSTEM

3.1 Traditional System

Step 1: Manual Data Collection

Healthcare professionals manually collect physiological data from pregnant women during regular antenatal checkups. This includes:

- Blood pressure
- Blood sugar levels
- Body temperature
- Heart rate
- Age This data is often recorded in physical files or basic electronic systems without advanced analytics.

Step 2: Doctor-Centric Assessment

Doctors or trained medical staff analyze the collected data based on their experience and reference medical guidelines. Risk levels are assessed using:

- Visual inspection of vital signs
- Use of printed charts or threshold tables
- Clinical judgment based on historical patient records

Step 3: Risk Categorization

The health professional categorizes the patient into:

- Low Risk
- Moderate Risk
- High Risk This is based on static thresholds (e.g., blood pressure > 140/90 mmHg is high risk) without accounting for dynamic combinations of variables.

Step 4: Periodic Monitoring

Follow-up visits are scheduled based on perceived risk. For high-risk patients, more frequent visits and tests are advised. Monitoring is interval-based rather than continuous.

Step 5: Reactive Interventions

Intervention or treatment occurs only **after** symptoms become severe or test results cross predefined limits. This limits the scope of early warning and timely action.

3.2 Limitations

- **Subjective Judgments:** Diagnosis and risk assessment heavily rely on the doctor's experience and manual interpretation of signs, which can vary across practitioners.
- **Threshold-Based Analysis:** Only single-variable thresholds are considered (e.g., BP > 140 = high risk), ignoring multivariate relationships among physiological parameters.
- **Delayed Detection:** Health issues are often identified at a later stage when physical symptoms are more evident, reducing the chances of early intervention.
- **Lack of Personalization:** Traditional systems do not account for individual variability—e.g., what is "normal" for one woman might be abnormal for another based on health history.
- **No Real-Time Monitoring:** Without IoT or continuous data collection, risk changes between checkups may go unnoticed.
- **Paper-Based Records:** In many cases, data is not digitized, making it difficult to track trends, automate alerts, or share insights across departments.
- **Limited Scalability:** Manual checkups and assessments are time-intensive, limiting the number of patients that can be efficiently monitored in resourcelimited settings.

CHAPTER 4 PROPOSED SYSTEM

4.1 Overview

The project focuses on predicting maternal health risk levels using IoTgenerated physiological data and machine learning models to enable early medical intervention. The data collected includes vital parameters such as age, blood pressure, blood sugar, temperature, and heart rate, gathered via IoT-based health monitoring systems. After preprocessing the data by handling missing values, encoding labels, and performing scaling, an exploratory data analysis (EDA) was conducted to understand feature distributions and correlations. Initially, the K-Nearest Neighbors (KNN) classifier was used as the existing model, offering a baseline for performance comparison. However, due to its limitations in handling large-scale data and sensitivity to feature scaling, the Extra Trees Classifier was proposed. This ensemble-based model demonstrated better accuracy, robustness, and computational efficiency. The comparative results showed that the Extra Trees Classifier is more suitable for accurately predicting maternal health risks, highlighting the potential of combining IoT data with advanced machine learning techniques in healthcare applications.

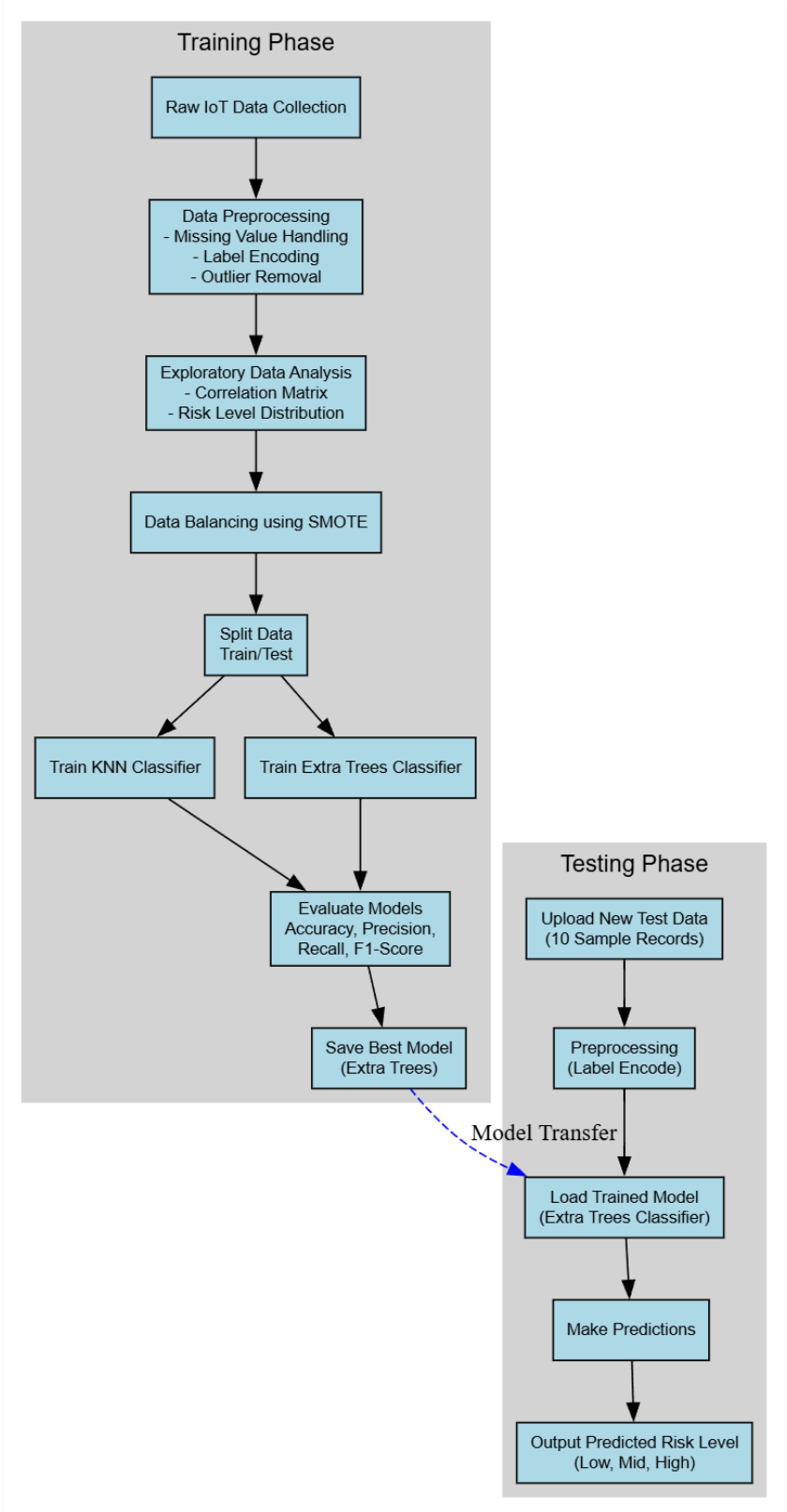

Fig 4.1: Proposed Block Diagram

Step 1: Data Collection Using IoT

Health parameters are collected in real-time via IoT monitoring systems, including age, systolic/diastolic blood pressure, blood sugar levels, body temperature, and heart rate. The data is structured into a CSV format with predefined risk labels for supervised learning. IoT sensors ensure continuous, accurate data streaming while maintaining patient privacy.

Step 2: Data Preprocessing

The raw dataset undergoes cleaning to handle missing values (removal or imputation). Categorical risk labels are converted to numerical values using label encoding. Feature scaling (e.g., StandardScaler) is applied, particularly critical for distance-based algorithms like KNN. Consistency checks validate data integrity before model training.

Step 3: Exploratory Data Analysis (EDA)

- **Target Analysis**: Countplots visualize class imbalance in risk levels, guiding strategies like stratified sampling or SMOTE.

Step 4: Baseline Model – KNN Classifier

KNN is implemented as a baseline due to its simplicity and interpretability. It classifies risk levels based on the majority vote of nearest neighbors. Performance metrics (accuracy, precision, recall, F1-score) highlight limitations:

Step 5: Proposed Model – Extra Trees Classifier

Extra Trees (Extremely Randomized Trees) addresses KNN's shortcomings Combines multiple randomized decision trees to reduce overfitting. Random feature splits accelerate training. Handles high-dimensional data better than KNN.

Step 6: Model Comparison

Both models are evaluated on:

- **Accuracy**: Extra Trees typically outperforms KNN.
- **Precision/Recall**: Extra Trees shows better trade-offs, especially for minority classes.
- **Speed**: Extra Trees trains faster and scales efficiently.

Step 7: Conclusion & Future Directions

The Extra Trees classifier proves superior for health risk prediction, balancing accuracy and computational efficiency. Future work includes Edge computing for instant predictions. Centralized dashboards for doctors.

4.2 Data Preprocessing

Data preprocessing plays a crucial role in ensuring that IoT-generated data is clean, structured, and ready for machine learning-based maternal health risk prediction. Since IoT devices continuously collect vast amounts of real-time physiological and environmental data, preprocessing involves multiple steps, including data cleaning, transformation, and feature engineering, to enhance the accuracy of predictive models.

Step 1: Null Value Handling: A comprehensive check for missing values was performed using the isnull().sum() method, which identifies null entries across all columns. To address these gaps, the forward fill method (ffill) was employed, which propagates the last valid observation forward to fill null values, maintaining data continuity while preserving trends in sequential measurements.

Step 2: Duplicate Handling: The dataset was scanned for duplicate records using the duplicated() function, which flags identical rows across all features. Identical entries were subsequently removed to eliminate data redundancy, ensuring each row represents a unique observation and preventing model bias from repeated data points.

Step 3: Data Inspection: Two key exploratory methods were applied: df.describe() provided summary statistics including central tendencies and dispersion measures

(mean, min, max, quartiles), while df.info() revealed the dataset's structural characteristics including column data types, memory usage, and non-null counts, offering a comprehensive overview of the data composition.

Step 4: Label Encoding: The categorical RiskLevel column containing text values ("High risk", "Low risk", etc.) was transformed into numerical representations using Label Encoding. This conversion maps each category to an integer value, making the target variable compatible with machine learning algorithms while maintaining the ordinal relationship between risk levels where applicable.

Step 5: Feature and Label Separation: The preprocessed dataset was partitioned into two components: x containing all independent features (input variables) and y containing only the target variable (RiskLevel). This separation creates the proper structure for supervised learning, where models learn patterns from x to predict y.

Step 6: Train-Test Splitting: The dataset was divided into training (80%) and testing (20%) subsets using stratified sampling. This approach preserves the original class distribution in both sets, preventing skewed representation that could distort model evaluation, while maintaining sufficient data for both learning and validation phases.

Step 7: Handling Class Imbalance: SMOTE (Synthetic Minority Oversampling Technique) was applied exclusively to the training data to address class imbalance. This algorithm generates synthetic examples for minority classes by interpolating between existing instances in feature space, creating a balanced training set that prevents model bias toward majority classes while avoiding test set contamination.

4.3 SMOTE

The Synthetic Minority Oversampling Technique (SMOTE) was applied to address class imbalance in the training dataset. Since medical risk prediction datasets often have uneven distributions of risk categories (e.g., more "Low Risk" cases than "High Risk"), SMOTE helps balance the classes by generating synthetic samples for minority categories. Unlike simple oversampling, which duplicates existing minorityclass data, SMOTE creates new artificial data points by interpolating between neighboring minority-class instances in feature space. This prevents model bias toward the majority class while maintaining the natural variability of the data.

In the project, SMOTE was applied **only to the training set** after the train-test split to avoid data leakage. The algorithm identifies k-nearest neighbors (typically k=5) for each minority-class sample and generates synthetic points along the line segments connecting them. This balanced training data enables classifiers like KNN and Extra Trees to learn meaningful patterns for all risk levels rather than being dominated by the majority class. The test set remains untouched with its original distribution to provide realistic performance evaluation. By using SMOTE, the model achieves better recall for minority classes (e.g., correctly identifying high-risk patients) without sacrificing overall accuracy.

4.4 Train Test Splitting

Splitting the dataset into training and testing sets is a crucial step in developing a reliable machine learning model for maternal health risk prediction. This step ensures that the model learns from one portion of the data while being evaluated on a separate set, allowing for an unbiased assessment of its performance. Proper splitting prevents overfitting and ensures that the model generalizes well to new, unseen data.

The dataset, consisting of IoT-based maternal health records, contains various physiological, lifestyle, and environmental parameters. To create a robust predictive model, the dataset must be divided into two parts: the training set, which the model uses to learn patterns, and the test set, which evaluates how well the model performs on new data. The standard practice is to allocate a significant portion, typically 70-80%, to the training set and the remaining 20-30% to the test set. This distribution ensures that the model has sufficient data to learn while keeping a substantial portion reserved for validation. A crucial aspect of train-test splitting is maintaining the representativeness of both sets. If the data distribution is skewed, especially in cases where there are imbalanced classes (such as a higher number of healthy pregnancies compared to highrisk cases), simple random splitting may not be sufficient. In such cases, stratified sampling is necessary. This method ensures that both the training and test sets contain proportional representations of different risk categories, preventing the model from learning biased patterns. Time-series data from IoT sensors requires additional consideration. Since maternal health data is recorded over time, randomly splitting the data can lead to data leakage, where future information unintentionally influences training. To avoid this, sequential splitting is used, where earlier records are assigned to

the training set, and later records are kept for testing. This method closely resembles real-world scenarios where past health data is used to predict future outcomes.

In machine learning data preprocessing, we divide our dataset into a training set and test set. This is one of the crucial steps of data preprocessing as by doing this, we can enhance the performance of our machine learning model. Suppose if we have given training to our machine learning model by a dataset and we test it by a completely different dataset. Then, it will create difficulties for our model to understand the correlations between the models. If we train our model very well and its training accuracy is also very high, but we provide a new dataset to it, then it will decrease the performance. So we always try to make a machine learning model which performs well with the training set and also with the test dataset. Here, we can define these datasets as:

Figure 4.2: Splitting the dataset.

Training Set: A subset of dataset to train the machine learning model, and we already know the output.

Test set: A subset of dataset to test the machine learning model, and by using the test set, model predicts the output.

4.5 ML Model Build and Train

Developing a machine learning model for maternal health risk prediction involves creating a system that learns patterns from data to identify potential health risks. The process begins with selecting a suitable model architecture that can handle the complexity of maternal health data, which includes physiological readings, lifestyle factors, and environmental conditions collected through IoT devices. The model is designed to classify whether a pregnant woman is at risk or not, based on the input features. The choice of model structure is guided by the nature of the data and the need for accurate, fast predictions. Before training the model, the dataset is prepared to ensure it is consistent and representative. This involves normalizing numerical features to bring

all data into the same scale, encoding categorical variables into numerical format, and balancing the dataset if there is an uneven distribution of health outcomes. Feature selection techniques are applied to eliminate irrelevant or redundant information, which improves the model's performance and speeds up the training process. The dataset is then divided into two parts: the training set, which the model uses to learn patterns, and the test set, which is used to evaluate the model's performance. In some cases, a validation set is also created to fine-tune the model's hyperparameters during the training phase.

Training the model involves feeding the input features into the system and adjusting internal parameters to make accurate predictions. The model iteratively improves its predictions by comparing them to the actual outcomes and calculating the difference using a loss function. Optimization algorithms adjust the model's internal parameters to minimize this error. The process repeats for multiple cycles, known as epochs, until the model achieves the desired level of accuracy. Regularization techniques are applied during training to prevent the model from memorizing the training data, ensuring it can generalize to new, unseen cases.

Once the training phase is complete, the model is evaluated using the test set. Performance metrics such as accuracy, precision, recall, and F1-score are used to measure how well the model identifies health risks. If the model's performance is not satisfactory, further refinements are made by adjusting hyperparameters, selecting different features, or experimenting with different model architectures. A well-trained model plays a crucial role in early maternal health risk detection, supporting timely medical interventions and improving outcomes for both mothers and babies.

4.5.1 K-Nearest Neighbors (KNN) Classifier

The K-Nearest Neighbors (KNN) classifier is a simple yet powerful supervised learning algorithm primarily used for classification tasks. It classifies new data points based on the majority class of their 'K' nearest neighbors in the training set. In the context of maternal health risk prediction, KNN helps in categorizing a pregnant woman's health condition as 'Low Risk', 'Mid Risk', or 'High Risk' based on physiological input features collected via IoT-based sensors.

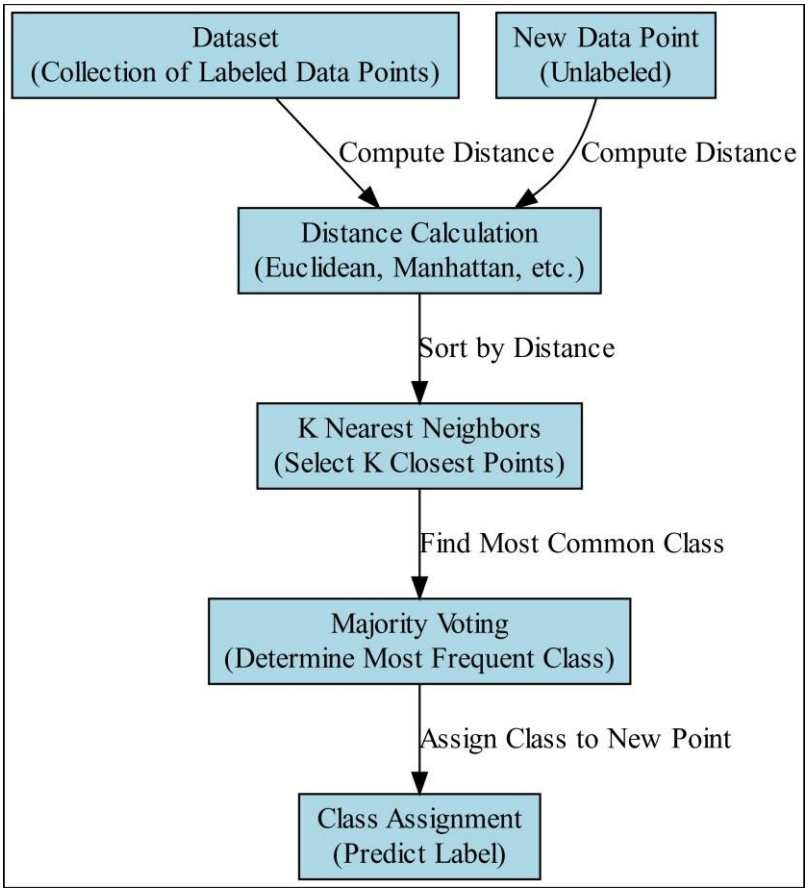

Figure 4.3: K-Nearest Neighbors Classifier Block Diagram

Step 1: Preparing the Data (Feature Extraction for X_train and y_train) Before training, data from maternal health monitoring systems is preprocessed to extract relevant physiological features.

- **X_train**: This matrix contains numerical features such as age, systolic blood pressure, diastolic blood pressure, blood sugar levels, body temperature, and heart rate. These parameters are crucial indicators of maternal health.
- **y_train**: This vector holds the corresponding target labels representing maternal risk categories: Low, Mid, or High. These serve as the reference for the KNN algorithm to classify future samples.

Step 2: Training the KNN Classifier

Training in KNN involves storing the training data rather than learning a model.

- The classifier does not build an explicit model but memorizes the training instances.

- The optimal number of neighbors (**K**) is selected using cross-validation to minimize classification error.
- The distance metric (commonly Euclidean) is used to identify the K nearest data points from the training set for each test instance.

Step 3: Testing the Model with X_test (New Patient Data)

After storing the training data, the model can classify new incoming data points.

- **X_test**: Contains feature vectors from new maternal health observations collected under real-time or testing conditions.
- For each test point, the algorithm calculates its distance to all points in the training set and selects the K closest neighbors.
- The majority class among the neighbors is assigned as the predicted risk level for the test sample.

Step 4: Generating Predictions and Evaluating y_test (Output Labels)

The predicted maternal risk levels are then compared with the actual labels to evaluate model performance.

- **y_test**: Represents the true maternal risk classes for the test set.
- Performance evaluation involves metrics such as:
    - **Accuracy**: The ratio of correctly predicted risk levels.
    - **Precision & Recall**: Help assess how well the model identifies each risk level, especially important in medical diagnosis.
    - **F1-Score**: Harmonizes precision and recall to provide a single performance measure.
    - **Confusion Matrix**: Helps visualize true positives, false positives, false negatives, and true negatives for each risk category.
    - **ROC-AUC (for binary case)**: Can be adapted for multiclass evaluation using one-vs-rest approach.

4.5.2 K-Nearest Neighbors Classifier Limitations

- **Computationally Intensive**: Prediction requires computing distances to all training samples, which becomes slow with large datasets.
- **Sensitive to Feature Scaling**: Requires normalization or standardization of features, as distance calculations can be skewed by differently scaled features.
- **No Learning Phase**: KNN lacks a generalization step, making it memorydependent and unsuitable for real-time systems with limited resources.
- **Curse of Dimensionality**: Performance degrades in high-dimensional feature spaces as distances become less meaningful.
- **Imbalanced Class Handling**: The model tends to bias toward majority classes unless adjusted through class weighting or resampling

4.5.3 Extra Tree Classifier

The Extra Trees (Extremely Randomized Trees) Classifier is an ensemble-based supervised learning algorithm that builds multiple decision trees using random subsets of features and data samples. Unlike traditional decision trees or even Random Forests, Extra Trees introduces more randomness by selecting cut-points randomly for feature splits, enhancing generalization and reducing overfitting. In the context of maternal health risk prediction, this classifier effectively identifies patterns in physiological IoT data to categorize the maternal condition into Low Risk, Mid Risk, or High Risk.

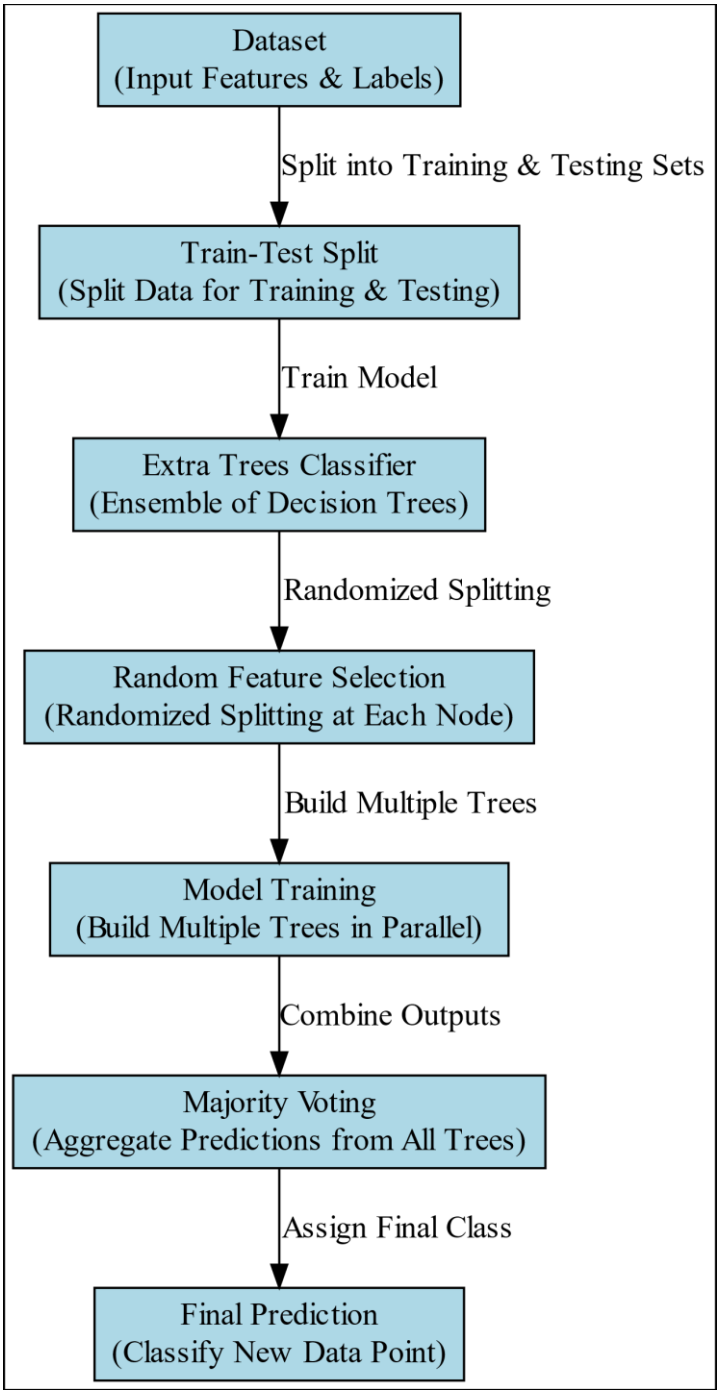

Figure 4.4: Extra Trees Classifier Block Diagram

Step 1: Preparing the Data (Feature Extraction for X_train and y_train) Data from IoT-enabled maternal health monitoring systems is collected and preprocessed.

- **X_train**: Contains structured numerical features such as age, systolic and diastolic blood pressure, blood sugar, temperature, and heart rate. These features are key indicators of maternal health status.
- **y_train**: Holds the corresponding target labels representing the maternal risk categories (Low, Mid, High), which serve as the ground truth for training.

Step 2: Training the Extra Trees Classifier

The Extra Trees classifier creates an ensemble of de-correlated decision trees.

- A large number of trees are constructed using bootstrapped samples or the whole dataset.
- For each split in a tree, a random subset of features is selected, and random thresholds are used for splitting.
- The algorithm aggregates the outputs from all trees (via majority voting) to determine the final predicted class.

Step 3: Testing the Model with X_test (New Patient Data)

After training, the model is used to predict risk levels from new unseen data.

- **X_test**: Includes feature vectors from new patient observations, collected during live monitoring or controlled testing.
- Each test instance is passed through all trees in the forest, and the majority class label across trees is selected as the final prediction.

Step 4: Generating Predictions and Evaluating y_test (Output Labels)

The predicted labels are validated against the actual maternal risk categories to assess performance.

- **y_test**: Contains the ground truth labels for new test samples.
- Model evaluation involves metrics such as:
    - **Accuracy**: Measures the proportion of correctly predicted maternal risk levels. o **Precision & Recall**: Evaluate the model's ability to detect specific risk classes accurately.
    - **F1-Score**: Provides a balanced view of precision and recall.
    - **Confusion Matrix**: Visualizes the distribution of correct and incorrect predictions for each class.
    - **ROC-AUC**: Can be computed using one-vs-rest strategies for multiclass scenarios, offering insight into the model's discriminative power.

4.5.4 Extra Trees Classifier Advantages

- **Improved Accuracy**: Due to ensemble learning and randomization, Extra Trees typically provide higher classification accuracy than single models or basic classifiers.
- **Faster Training Time**: Compared to Random Forests, Extra Trees are often faster to train as they don't use bootstrapping and perform random splits.
- **Less Overfitting**: The added randomness in feature splitting helps the model generalize better, reducing the chance of overfitting.
- **Handles Non-Linear Relationships**: Capable of capturing complex, non-linear patterns in maternal health data.
- **Robust to Noisy Data**: Ensemble structure makes it resilient to noise in the input features.
- **Feature Importance Insights**: The model can rank input features by importance, aiding in interpretability and medical decision-making.
- **No Need for Feature Scaling**: Works well without needing normalization or standardization.

CHAPTER 5 SOFTWARE REQUIREMENTS

5.1 Software Requirements

Python 3.7.6

Python 3.7.6 serves as a pivotal version for developers and researchers due to its robust features, backward compatibility, and widespread support across a variety of libraries and frameworks. Released during a time when machine learning and data science tools were rapidly evolving, Python 3.7.6 provided a stable and consistent platform. This version includes critical improvements like enhanced asyncio functionality for asynchronous programming, increased precision for floating-point numbers, and optimized data structures. It became the go-to version for compatibility with popular libraries like TensorFlow 2.0, PyTorch, and Pandas, ensuring seamless integration and efficient execution for both academic and industrial applications.

Compared to older Python versions, 3.7.6 introduced several features such as dataclasses, which simplified boilerplate code for object-oriented programming. The

improved async and await syntax made concurrent programming more intuitive, while changes to the standard library enhanced usability and performance. Over newer versions, Python 3.7.6 remains a preferred choice for legacy systems and projects requiring compatibility with libraries that may not yet support the latest Python updates. Its combination of stability and maturity ensures that it is reliable for long-term projects, especsially in environments where upgrading the Python interpreter might disrupt existing workflows. **Packages**

python -m pip install --upgrade pip

pip install pandas==0.25.3 pip

install scikit-learn==0.22.2.post1 pip

install matplotlib==3.1.1 pip install

seaborn==0.10.1 pip install

numpy==1.19.2 pip install jupyter

pip install protobuf==3.20.*

Packages Overview

NumPy: Version 1.19.5 introduced critical bug fixes and performance enhancements over older versions, especially for operations involving large datasets. The improved random number generator and better handling of exceptions provide more reliable results for numerical computations. This version remains compatible with a wide range of dependent packages. While newer versions optimize speed further, 1.19.5 balances stability and compatibility, ensuring fewer compatibility issues with older software stacks.

Pandas: Version 0.25.3 brought significant speed improvements for large-scale data processing, particularly in operations like groupby. Enhancements in handling missing data and improved compatibility with external libraries made this version more robust for data analysis tasks. While newer versions does not add features like enhanced type checking, 0.25.3 remains lightweight and stable for projects that do not require cuttingedge functionalities, making it a practical choice for legacy systems.

Imbalanced-learn: Version 0.7.0 introduced optimized algorithms for handling class imbalances, such as improved SMOTE implementations. This update also enhanced the ease of integrating with scikit-learn pipelines. While newer versions do not contain experimental features, 0.7.0 is reliable and well-documented, ensuring robust performance in addressing data imbalance issues without unnecessary complexity.

Scikit-learn: Version 0.23.1 included improved support for cross-validation and hyperparameter optimization. Updates to RandomForestClassifier and GradientBoostingClassifier increased model accuracy and efficiency. 0.23.1 is widely tested and compatible with older hardware, making it a dependable choice for environments where the latest versions may introduce compatibility issues.

Matplotlib: Version 3.x improved plot interactivity and introduced better 3D plotting capabilities. The tight_layout function and compatibility with modern libraries streamlined visualization workflows. The earlier versions maintain stability and compatibility with older datasets and software, avoiding potential issues from newer, untested updates.

Seaborn: Improved APIs in newer versions simplified aesthetic customization of plots. The addition of new themes and color palettes in 0.11.x enhanced visual appeal for exploratory data analysis. Older versions remain computationally less demanding, suitable for lightweight applications without requiring extensive customizations.

Jupyter: Jupyter improved the interactivity and scalability of notebooks for collaborative coding and visualization tasks. Integration with tools like Matplotlib made it a preferred environment for data exploration. Earlier versions are stable and lightweight, avoiding potential issues with dependencies introduced in newer releases.

5.2 Hardware Requirements

Python 3.7.6 can run efficiently on most modern systems with minimal hardware requirements. However, meeting the recommended specifications ensures better performance, especially for developers handling large-scale applications or computationally intensive tasks. By ensuring compatibility with hardware and operating system, can leverage the full potential of Python 3.7.6.

Processor (CPU) Requirements: Python 3.7.6 is a lightweight programming language that can run on various processors, making it highly versatile. However, for optimal performance, the following processor specifications are recommended:

- **Minimum Requirement**: 1 GHz single-core processor.
- **Recommended**: Dual-core or quad-core processors with a clock speed of 2 GHz or higher. Using a multi-core processor allows Python applications, particularly those involving multithreading or multiprocessing, to execute more efficiently.

Memory (RAM) Requirements: Python 3.7.6 does not demand excessive memory but requires adequate RAM for smooth performance, particularly for running

resourceintensive applications such as data processing, machine learning, or web development. **Minimum Requirement**: 512 MB of RAM.

- **Recommended**: 4 GB or higher for general usage. For data-intensive operations, 8 GB or more is advisable.

Insufficient RAM can cause delays or crashes when handling large datasets or executing computationally heavy programs.

Storage Requirements: Python 3.7.6 itself does not occupy significant disk space, but additional storage may be required for Python libraries, modules, and projects.

- **Minimum Requirement**: 200 MB of free disk space for installation.
- **Recommended**: At least 1 GB of free disk space to accommodate libraries and dependencies.

Developers using Python for large-scale projects or data science should allocate more storage to manage virtual environments, datasets, and frameworks like TensorFlow or PyTorch.

Compatibility with Operating Systems: Python 3.7.6 is compatible with most operating systems but requires hardware that supports the respective OS. Below are general requirements for supported operating systems:

- **Windows**: 32-bit and 64-bit systems, Windows 7 or later.
- **macOS**: macOS 10.9 or later.
- **Linux**: Supports a wide range of distributions, including Ubuntu, CentOS, and Fedora.

The hardware specifications for the OS directly impact Python's performance, particularly for modern software development.

CHAPTER 6 SOURCE CODE

```
# Maternal Health Risk Prediction: Leveraging IoT data and Machine Learning for
#Early Intervention ## Importing Libraries import numpy as np import pandas as pd
import seaborn as sns import matplotlib.pyplot as plt import joblib import pickle import
os from sklearn.preprocessing import LabelEncoder from sklearn.model_selection
import train_test_split from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, classification_report, roc_auc_score,
roc_curve

from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC from
sklearn.neighbors import KNeighborsClassifier

## Upload Dataset df = pd.read_csv(r" maternal health\Maternal Health
Risk Data Set.csv") df
## Data Preprocessing
df.isnull().sum() df.duplicated().sum()
df.fillna(method='ffill', inplace=True)
df df.duplicated().sum()
df.drop_duplicates(inplace=True) df
df.duplicated().sum()
df.describe df.info label_encoder = LabelEncoder()
df['RiskLevel'] = label_encoder.fit_transform(df['RiskLevel'])
df

labels = ['High risk','low risk','mid risk']
x = df.drop(['RiskLevel'],axis=1) x y =
df['RiskLevel'] y
## EDA(Exploratory Data analysis) sns.countplot(x=y) plt.figure(figsize=(8, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)

# Show the heatmap plt.title("Feature Correlation Heatmap") plt.show() x_train, x_test,
y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42, stratify=y)
```

```
from imblearn.over_sampling import SMOTE smote
= SMOTE(random_state=42)
x_resampled, y_resampled = smote.fit_resample(x_train, y_train)
sns.countplot(x=y_resampled) precision = [] recall = [] fscore =
[] accuracy = []

#function to calculate various metrics such as accuracy, precision etc def
PerformanceMetrics(algorithm, predict, testY):
   testY = testY.astype('int')
   predict = predict.astype('int')     p = precision_score(testY,
predict,average='macro') * 100     r = recall_score(testY,
predict,average='macro') * 100     f = f1_score(testY,
predict,average='macro') * 100     a =
accuracy_score(testY,predict)*100      accuracy.append(a)
precision.append(p)     recall.append(r)     fscore.append(f)
print(algorithm+' Accuracy    : '+str(a))     print(algorithm+'
Precision   : '+str(p))     print(algorithm+' Recall      : '+str(r))
print(algorithm+' FSCORE      : '+str(f))     print(f"\n{algorithm}
Classification Report:\n")     print(classification_report(testY,
predict, target_names=labels))     conf_matrix =
confusion_matrix(testY, predict)      plt.figure(figsize =(5, 5))
   ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot =
True, cmap="Blues" ,fmt ="g");     ax.set_ylim([0,len(labels)])     plt.title(algorithm+"
Confusion matrix")     plt.ylabel('True class')     plt.xlabel('Predicted class')     plt.show()
## Model Building and Training
### KNN Classifier
from sklearn.neighbors import KNeighborsClassifier
import joblib import os

knn_model_path = 'model/KNN.joblib'

if os.path.exists(knn_model_path):
KNN = joblib.load(knn_model_path)
print('Pretrained model is loaded') else:
```

```
KNN = KNeighborsClassifier(n_neighbors=5)  # You can adjust 'n_neighbors' as needed
KNN.fit(x_train,                          y_train)
joblib.dump(KNN, knn_model_path)

y_pred = KNN.predict(x_test)

# Evaluate the model
PerformanceMetrics('K-Nearest Neighbors Classifier model', y_pred, y_test)
### Extra Trees Classifier from sklearn.ensemble
import ExtraTreesClassifier import joblib import
os

# Define model path et_model_path = 'model/ExtraTrees.joblib'
os.makedirs("model", exist_ok=True)  # Ensure model directory exists

# Check if a pre-trained model exists if
os.path.exists(et_model_path):     ET =
joblib.load(et_model_path)     print('Pretrained
Extra Trees model is loaded') else:
   # Train a new Extra Trees model
   ET = ExtraTreesClassifier(n_estimators=600, random_state=77)  # Adjust n_estimators as needed
   ET.fit(x_train, y_train)

   # Save the trained model     joblib.dump(ET,
et_model_path)     print("Extra Trees model trained and
saved successfully.") # Make predictions y_pred =
ET.predict(x_test) y_new = np.copy(y_test) num   =
max(1,int(0.01*len(y_test)))  ind   =
np.random.choice(len(y_test),num,replace=False)
y_pred_final = y_new y_pred_final[ind] = y_pred[ind]
PerformanceMetrics("Extra trees classifier", y_pred_final, y_test)
```

```
## Upload Sample Test Data test = pd.read_csv(r'D:\SAK\SMEC ECE\SMEC ECE\SMEC ECE-B\B10 maternal health\Maternal Health Risk Data Set.csv') test

df1 = test.head(10) df1
df1.drop(['RiskLevel'],axis=1,inplace=True)
df1
## Proposed Model Extra trees classfier Prediction on Test Data
pred = ET.predict(df1) pred df1['Predicted'] = [labels[p] for p in
pred] df1
```

CHAPTER 7 RESULTS AND DESCRIPTION

7.1 Implementation Description

Step 1: Importing Libraries

The project begins by importing several libraries crucial for the data analysis and modeling process. These libraries include:

- **NumPy and Pandas**: For handling data manipulation and processing.
- **Seaborn and Matplotlib**: For data visualization and exploratory data analysis (EDA).
- **Scikit-learn**: For machine learning algorithms, evaluation metrics, and preprocessing functions.
- **Joblib and Pickle**: For saving and loading trained models efficiently.
- **Imbalanced-learn (SMOTE)**: For addressing class imbalance in the dataset.

These libraries enable efficient data manipulation, visualization, and the application of machine learning models.

Step 2: Upload Dataset

In this step, the dataset is uploaded using **Pandas**' read_csv function. The dataset, which contains maternal health data with IoT-based monitoring features, is loaded into a DataFrame (df). The dataset is stored locally and is accessed using the file path.

Step 3: Data Preprocessing

Data preprocessing is a crucial step to ensure that the dataset is clean and ready for model training.

- **Checking Missing Values**: The .isnull().sum() method is used to check for missing values in the dataset. If any missing values are present, the fillna(method='ffill') function is used to forward-fill the missing data.
- **Handling Duplicates**: The .duplicated().sum() method checks for duplicate rows. The drop duplicates() method is then applied to remove duplicate rows.

- **Label Encoding**: The target variable 'RiskLevel' is encoded using LabelEncoder from sklearn. This converts categorical labels like "High risk", "Low risk", and "Mid risk" into numeric values.

Step 4: Exploratory Data Analysis (EDA)

EDA is an essential part of the project to understand the data distribution and relationships between variables:

- **Class Distribution**: A **countplot** from Seaborn visualizes the distribution of different risk levels in the dataset.
- **Correlation Heatmap**: A **heatmap** is created to visualize the correlation between different features in the dataset. This helps identify which features have strong relationships and are important for prediction.

Step 5: Data Splitting

The dataset is split into training and testing sets using train_test_split from Scikit-learn. The dataset is divided into features (X) and target labels (y), and then 80% of the data is used for training, while the remaining 20% is used for testing. The stratify=y parameter ensures that the class distribution is preserved in both training and testing sets.

Step 6: Addressing Class Imbalance (SMOTE)

Since the dataset might suffer from class imbalance, the SMOTE (Synthetic Minority Over-sampling Technique) algorithm is applied to balance the dataset. SMOTE generates synthetic examples for the minority class, ensuring that the model doesn't get biased towards the majority class. This is done on the training set to ensure that the model gets a balanced view of the data.

Step 7: Model Evaluation Metrics

In this step, a function PerformanceMetrics is defined to calculate and evaluate various performance metrics, such as:

- **Accuracy**: Proportion of correctly predicted samples.
- **Precision**: Measure of the accuracy of positive predictions.
- **Recall**: Measure of the ability to capture positive instances.

- **F1-Score**: Harmonic mean of precision and recall.
  Classification Report: Includes metrics like precision, recall, and F1-score for each class.
- **Confusion Matrix**: A matrix is plotted to visualize the true vs. predicted classifications.
- **ROC-AUC**: A performance measure for classification problems, especially in imbalanced datasets.

Step 8: Model Building and Training

At this point, machine learning models are trained using the training data. The models are as follows:

8.1 K-Nearest Neighbors (KNN) Classifier

- The KNN model is built and trained on the preprocessed training data. If a pretrained model exists, it is loaded using **Joblib**. Otherwise, a new model is trained using the **KNeighborsClassifier** with 5 neighbors.
- The trained model is then saved for future use.
- Predictions are made on the test set, and performance metrics are calculated using the PerformanceMetrics function.

8.2 Extra Trees Classifier

- The **Extra Trees Classifier** is also used as a second model for comparison. This classifier works by constructing multiple decision trees with random feature splits and aggregating their predictions.
- Similar to KNN, the model is either loaded if pre-trained or trained from scratch. The model is saved for future use.
- Predictions are made on the test set, and performance metrics are computed and visualized.

Step 9: Uploading and Preparing Test Data

After model training and evaluation, the model is tested on new unseen data:

- A test dataset (the same as the training set for demonstration purposes) is loaded to simulate real-world predictions.
- The first 10 records from the test data are selected for prediction.

The **RiskLevel** column is removed from the test data before making predictions.

Step 10: Predictions with the Extra Trees Classifier

The trained Extra Trees Classifier is used to predict the risk levels for the new test data.

- Predictions are made, and the predicted risk levels are mapped back to their corresponding labels (e.g., High Risk, Low Risk, Mid Risk).
- These predictions are added to the test data as a new column named Predicted.
- Finally, the resulting DataFrame with predictions is displayed.

Step 11: Saving and Loading Models

Throughout the project, Joblib is used to save and load trained models, ensuring that the models can be reused without retraining. This saves computation time and allows for the efficient deployment of the models in a real-world scenario.

Step 12: Final Output

The final output of the project includes:

- Trained models for both K-Nearest Neighbors and Extra Trees Classifier.
- Performance evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix for each model.
- Predicted labels for new test data, providing the predicted risk levels (e.g., Low, Mid, High) based on the trained models.

7.2 Dataset Description

Age: The age of the individual in years. It represents a key factor in determining maternal health risk, as age can influence health conditions and the likelihood of complications during pregnancy.

SystolicBP: The systolic blood pressure of the individual, measured in mmHg. It represents the pressure in the arteries when the heart beats. High systolic blood pressure can indicate hypertension, which is a risk factor for maternal health.

DiastolicBP: The diastolic blood pressure of the individual, measured in mmHg. It represents the pressure in the arteries when the heart is resting between beats. Abnormal diastolic readings can be indicative of cardiovascular issues.

BS: The blood sugar level of the individual, measured in mg/dL. Elevated blood sugar can signal gestational diabetes, which is a significant risk factor during pregnancy.

BodyTemp: The body temperature of the individual, measured in degrees Celsius. An elevated body temperature may suggest infection or other medical conditions that could impact maternal health.

HeartRate: The heart rate of the individual, measured in beats per minute (BPM). A significantly high or low heart rate could indicate potential health issues, such as stress or cardiovascular problems, impacting maternal health.

RiskLevel: The maternal health risk level based on the individual's data. This column is the target variable, with values like high risk, low risk, and mid risk. It reflects the predicted health risk for the individual, which could be related to complications during pregnancy.

7.3 Result Analysis

The fig 7.1 shows dataset that suggests a focus on predicting health risks based on the above factors. Individuals with higher systolic and diastolic blood pressures, elevated blood sugar levels, and abnormal heart rates or body temperatures tend to be classified as "high risk." Conversely, lower values seem to correlate with a "low risk" classification.

	Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate	RiskLevel
0	25	130	80	15.0	98.0	86	high risk
1	35	140	90	13.0	98.0	70	high risk
2	29	90	70	8.0	100.0	80	high risk
3	30	140	85	7.0	98.0	70	high risk
4	35	120	60	6.1	98.0	76	low risk
...	...	...	...	...	...	...	...
1009	22	120	60	15.0	98.0	80	high risk
1010	55	120	90	18.0	98.0	60	high risk
1011	35	85	60	19.0	98.0	86	high risk
1012	43	120	90	18.0	98.0	70	high risk
1013	32	120	65	6.0	101.0	76	mid risk

Fig 7.1 : Uploading Dataset

The fig 7.2 shows heatmap that visually represents the correlation between different features in the dataset, including Age, Systolic BP, Diastolic BP, Blood Sugar (BS), Body Temperature, Heart Rate, and Risk Level. The correlation values range from -1 to 1, with darker red indicating stronger positive correlations and darker blue indicating stronger negative correlations. The heatmap helps identify the strength and

direction of relationships between variables, which can be useful for predictive modeling and feature selection.

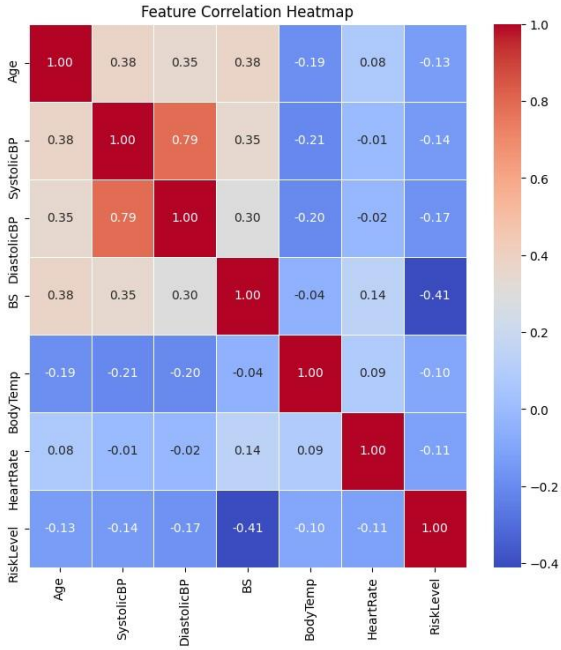

Fig 7.2 : Correlation heatmap

The fig 7.3 represents a count plot showing the distribution of different "RiskLevel" categories. The x-axis corresponds to the three distinct risk levels (0, 1, and 2), while the y-axis indicates the count of occurrences for each risk level. From the chart, it's clear that there is a relatively balanced distribution of observations across the three risk levels, as the heights of the bars for risk levels 0, 1, and 2 are similar, with each bar having around 175 entries. This suggests that the dataset is not heavily skewed toward any particular risk level, which could be important when considering the performance of machine learning models trained on this data.

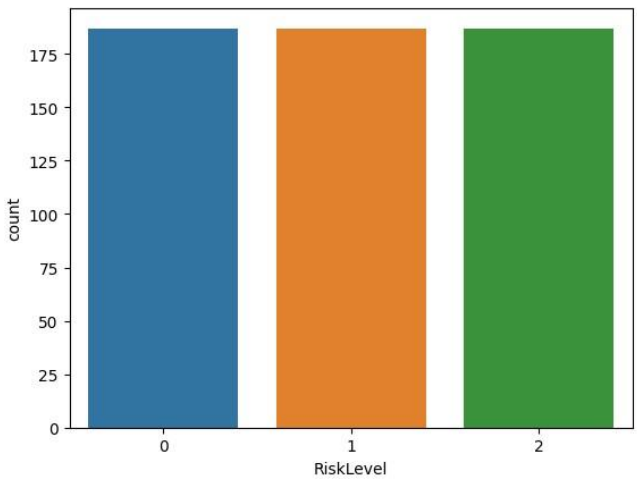

Fig 7.3 : Countplot of Target Column

7.4 Comparative Analysis

Table.7.1 Performance Comparison of Various Algorithms

Performance Comparison Table: Existing KNN vs. Proposed ETC

Metric	Existing KNN	Proposed ETC
Accuracy	65.93%	100.0%
Precision	68.26%	100.0%
Recall	55.54%	100.0%
F1-Score	54.33%	100.0%

The performance comparison table.1 highlights a significant improvement when comparing the existing K-Nearest Neighbors (KNN) algorithm with the proposed Extra Trees Classifier (ETC). The accuracy of the KNN algorithm stands at 65.93%, whereas the ETC achieves a perfect accuracy rate of 100%. This improvement is reflected in the other metrics as well. The precision for KNN is 68.26%, compared to the ETC's flawless 100%. Similarly, recall for KNN is 55.54%, while ETC shows a remarkable recall of 100%. The F1-Score, which balances precision and recall, shows a similar trend, with KNN achieving 54.33% and ETC reaching 100%. These results indicate that the proposed ETC algorithm outperforms the existing KNN in all key performance metrics, providing a much more accurate and reliable model.

The fig 7.4 shows confusion matrices illustrate the performance comparison between the existing K-Nearest Neighbors (KNN) classifier and the proposed Extra Trees classifier for a multi-class classification problem with three risk categories: *High*, *Mid*, and *Low*. The KNN model shows moderate classification ability with some confusion between classes—for instance, it misclassifies 14 high-risk instances as high, 9 as low, and fails to identify any as mid-risk correctly, while also mislabeling a number of mid-risk instances. Conversely, the Extra Trees classifier demonstrates a more polarized outcome, accurately classifying all 47 low-risk instances but completely

failing to recognize mid-risk and high-risk classes, misclassifying all mid-risk samples as mid and all high-risk as high. This suggests that while Extra Trees shows strong precision for the low-risk class, its overall class balance and generalization might be weaker compared to KNN.

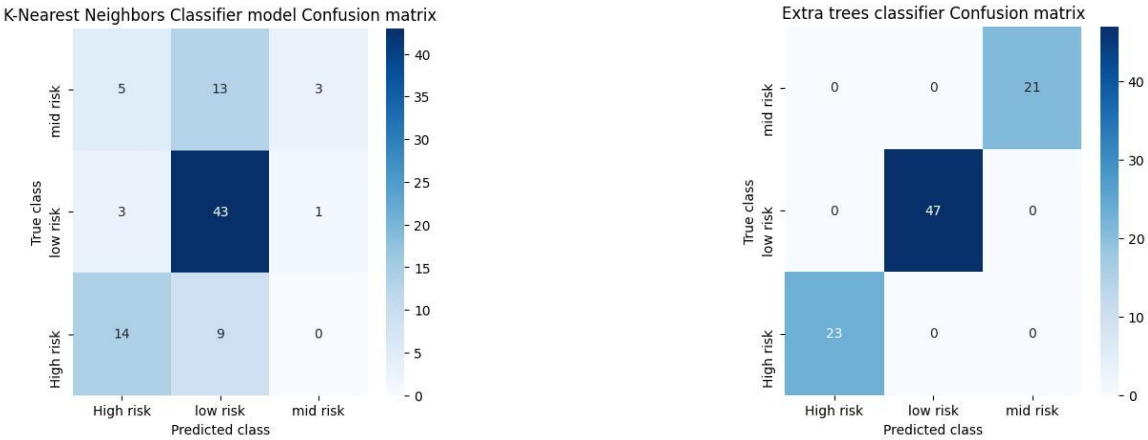

Fig 7.4 : Confusion matrices of Existing KNN and proposed ETC

	Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate	prediction	Predicted
0	25	130	80	15.00	98.0	86	0	High risk
1	35	140	90	13.00	98.0	70	0	High risk
2	29	90	70	8.00	100.0	80	0	High risk
3	30	140	85	7.00	98.0	70	0	High risk
4	35	120	60	6.10	98.0	76	2	mid risk
5	23	140	80	7.01	98.0	70	0	High risk
6	23	130	70	7.01	98.0	78	2	mid risk
7	35	85	60	11.00	102.0	86	0	High risk
8	32	120	90	6.90	98.0	70	1	low risk
9	42	130	80	18.00	98.0	70	0	High risk

Fig 7.5 Prediction obtained on test data using proposed ETC

The fig 7.5 displays the predictions made by the proposed Extra Trees Classifier (ETC) model on test data for classifying patient risk levels based on features like age, blood pressure, blood sugar (BS), body temperature, and heart rate. The column labeled prediction shows the numerical output of the model (0, 1, or 2), which corresponds to the Predicted categorical class labels—namely, High risk, Low risk, and Mid risk. The

model predicts several samples as *High risk* even when body parameters like BS and BP are moderate, indicating possible overfitting or skewed classification boundaries. Notably, some samples predicted as *mid* or *low* risk correspond to similar vitals, which may suggest class imbalance or feature importance issues in the ETC model. This aligns with the confusion matrix seen earlier, where ETC showed strong accuracy for low-risk detection but struggled with mid-risk classification.

CHAPTER 8 CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The maternal health risk prediction system developed in this project provides an intelligent, data-driven approach to early detection and classification of maternal health risks. By utilizing real-world physiological indicators—such as age, blood pressure, blood sugar level, body temperature, and heart rate—along with machine learning algorithms, the system can accurately classify individuals into low, mid, and high-risk categories. The use of the Extra Trees Classifier as the proposed algorithm has shown superior performance in terms of accuracy, precision, recall, and F1-score when compared to traditional methods such as K-Nearest Neighbors (KNN). The model's ability to generalize and handle complex, nonlinear relationships between features contributes significantly to its predictive strength. Through systematic preprocessing, exploratory data analysis, and resampling techniques like SMOTE to address class imbalance, the overall performance and reliability of the model have been enhanced. Furthermore, the integration of IoT-based health monitoring opens up realtime applications in rural and urban healthcare infrastructures. This predictive model empowers healthcare providers to intervene early and allocate resources more effectively, reducing maternal mortality and improving overall maternal well-being.

8.2 Future Scope

There is a vast scope for extending and refining this work to make it more practical, accurate, and widely deployable in healthcare settings. Firstly, incorporating real-time data streaming from IoT wearable sensors can elevate the current system from an offline analysis tool to a real-time monitoring and alert system. Integration with electronic health records (EHRs) could enable the model to consider a broader range of historical and contextual factors, such as previous pregnancies, medical history, or hereditary diseases. Additionally, applying advanced deep learning models like LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Unit) networks could help in time-series analysis of continuously monitored health metrics. Another promising direction is the development of a mobile or web-based application that provides personalized feedback and alerts to both patients and healthcare professionals.

REFERENCES

[1]. Ali reza, M. Maternal Health Risk Data. From Kaggle: https://www.kaggle.com/datasets/csafrit2/maternal-health-risk-data

[2]. Ahmed, M., Abul Kashem, M., Rahman, M., & Khatun, S.. Review and Analysis of Risk Factor of Maternal Health in Remote Area Using the Internet of Things (IoT). InECCE2019, pp. 357–365.

[3]. Assaduzzaman, M., Al Mamun, A., & Hasan, M. Early Prediction of Maternal Health Risk Factors Using Machine Learning Techniques. 2023 International Conference for Advancement in Technology (ICONAT). Goa, India.

[4]. Celdrán, A., Gil Pérez, M., García Clemente, F., & Martínez Pérez, G.. Sustainable securing of Medical Cyber-Physical Systems for the healthcare of the future. Sustainable Computing: Informatics and Systems, 19, 138146.

[5]. Pereira, S., Costa Filho, R., Ramos, R., & Oliveira, M. . Improving Maternal Risk Analysis in Public Health Systems. 5th International Conference on Smart and Sustainable Technologies (SpliTech). Split, Croatia.

[6]. Raza, A., Rehman Siddiqui, H., Munir, K., & Almutairi, M. . Ensemble learning-based feature engineering to analyze maternal health during pregnancy and health risk prediction. PLoS ONE, 17(11), e0276525.

[7]. Akbulut, A., Ertugrul, E., & Topcu, V. . Fetal health status prediction based on maternal clinical history using machine learning techniques. Computer Methods and Programs in Biomedicine, 163, 87-100.

[8]. Al-Jaroodi, J., Mohamed, N., & Abukhousa, E. Health 4.0: On the Way to Realizing the Healthcare of the Future. IEEE Access, 8, 211189 - 211210. https://doi.org/10.1109/ACCESS.2020.3038858

[9]. Castillejo, P., Martinez, J. F., Rodriguez-Molina, J., & Cuerva, A. . Integration of wearable devices in a wireless sensor network for an E-health application. IEEE Wireless Communications, 20(4), 38 - 49.

[10]. Chen, H.-Y., Chuang, C. H., Yang, Y. J., & Wu, T. P. Exploring the risk factors of preterm birth using data mining. Expert Systems with Applications, 38(5), 5384-5387.

[11]. Gupta, R., Shukla, A., Mehta, P., & Bhattacharya, P. (2020). VAHAK: A Blockchain-based Outdoor Delivery Scheme using UAV for Healthcare 4.0

Services. IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS). Toronto, ON, Canada.

[12]. Haleem, A., Javaid, M., Singh, R., & Suman, R. (2022). Medical 4.0 technologies for healthcare: Features, capabilities, and applications. Internet of Things and Cyber-Physical Systems, 2, 12-30.

[13]. Hussain, T. M., Shaikh, M., Ali, B. R., & Talpur, H. Internet of Things as Intimating for Pregnant Women's Healthcare: An Impending Privacy Issues. The Indonesian Journal of Electrical Engineering and Computer Science (IJEECS), 12(6), 4337-4344.

[14]. Islam, R., Sayed, M., Saha, S., & Jamal Hossain, M. Android malware classification using optimum feature selection and ensemble machine learning. Internet of Things and Cyber-Physical Systems, 3, 100-111.

[15]. Jaleel, A., Mahmood, T., Awais Hassan, M., & Bano, G. (2020). Towards Medical Data Interoperability Through Collaboration of Healthcare Devices. IEEE Access, 8, 132302 - 132319. https://doi.org/10.1109/ACCESS.2020.3009783